CS52 · AI & ML · CIE-1 · PREDICTED QUESTIONS

Unit I: Agents, Search & Logic

PEAS · A* on 8-puzzle · Propositional-logic validity · Learning vs goal-based agent · CNF conversion · Environment types · Choosing a search strategy

SURE SHOT named in the insider list / CR list or repeated from last year — learn word-perfect

VERY LIKELY came last year or is the natural variant

IMPORTANT TOPIC fits the "scenario" hint — study the idea

I Unit I — Agents, Search & Logic (theory + one big problem)

Q1. Give the PEAS description for a self-driving car (or any given agent) and classify its environment. SURE SHOT

Why: "PEAS" is on the insider list; last year's Q1 had an agent question. Expect a fresh scenario: self-driving car, food-delivery drone, medical-diagnosis system, robot vacuum, chess bot, smart home assistant.

DEFINITION (1 MARK)

PEAS = **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors. It specifies the task environment of an agent — the first step in designing any agent. Performance = how success is judged; Environment = where it works; Actuators = how it acts; Sensors = how it perceives.

PEAS	Self-driving car (automated taxi)
Performance	Safe (no accidents), fast / shortest time, legal (obeys traffic rules), comfortable trip, fuel-efficient, maximise profit / passenger satisfaction
Environment	Roads, other vehicles, pedestrians, cyclists, traffic signals, weather, potholes, passengers
Actuators	Steering, accelerator, brake, gear, horn, indicators, headlights, display / speaker to passenger
Sensors	Cameras, LIDAR, RADAR, GPS, speedometer, odometer, accelerometer, engine sensors, microphone / touchscreen

Environment type (1 mark): partially observable (can't see around corners), multi-agent (other drivers), stochastic (unpredictable traffic), **sequential** (each action affects the future), **dynamic** (world changes while deciding), **continuous** (speed, position), known. → the hardest kind of environment.

Agent	Performance	Environment	Actuators	Sensors
Medical diagnosis	Healthy patient, low cost, fewer lawsuits	Patient, hospital, staff	Display questions, tests, diagnoses, treatments	Keyboard entry of symptoms, test results, patient answers
Robot vacuum	Cleanliness, battery use, time, no damage	Rooms, floor, furniture, dirt, people, pets	Wheels, brushes, suction, bumper	Dirt sensor, bump sensor, cliff sensor, camera
Food-delivery drone	On-time delivery, safety, battery, intact food	Sky, buildings, weather, birds, customers	Propellers, gripper / drop mechanism	GPS, camera, altimeter, wind sensor
Chess bot (with clock)	Win / draw / loss, time used	Board, opponent, clock	Move pieces (display move)	Board state, clock
Wumpus-world agent	+1000 gold out, −1000 death, −1/step, −10 arrow	4×4 cave, wumpus, pits, gold	Forward, TurnLeft/Right, Grab, Shoot, Climb	Stench, Breeze, Glitter, Bump, Scream
English tutor	Students' test scores	Students, testing agency	Exercises, suggestions, corrections	Typed words / speech

REMEMBER IT

"P-E-A-S = Purpose, Every-thing around, Arms, Senses." Actuators = what it **does** (verbs: steer, brake). Sensors = what it **feels** (nouns: camera, GPS). Never put "camera" under actuators — that's the most common mistake.

Q2. Apply A* to solve the 8-puzzle (g = depth, h = number of misplaced tiles).

SURE SHOT

Why: last year's Q2a (8 marks) + "A*" on the insider list. Same method works for any start/goal they give.

Start (last year's paper)

2	8	3
1	6	4
7		5

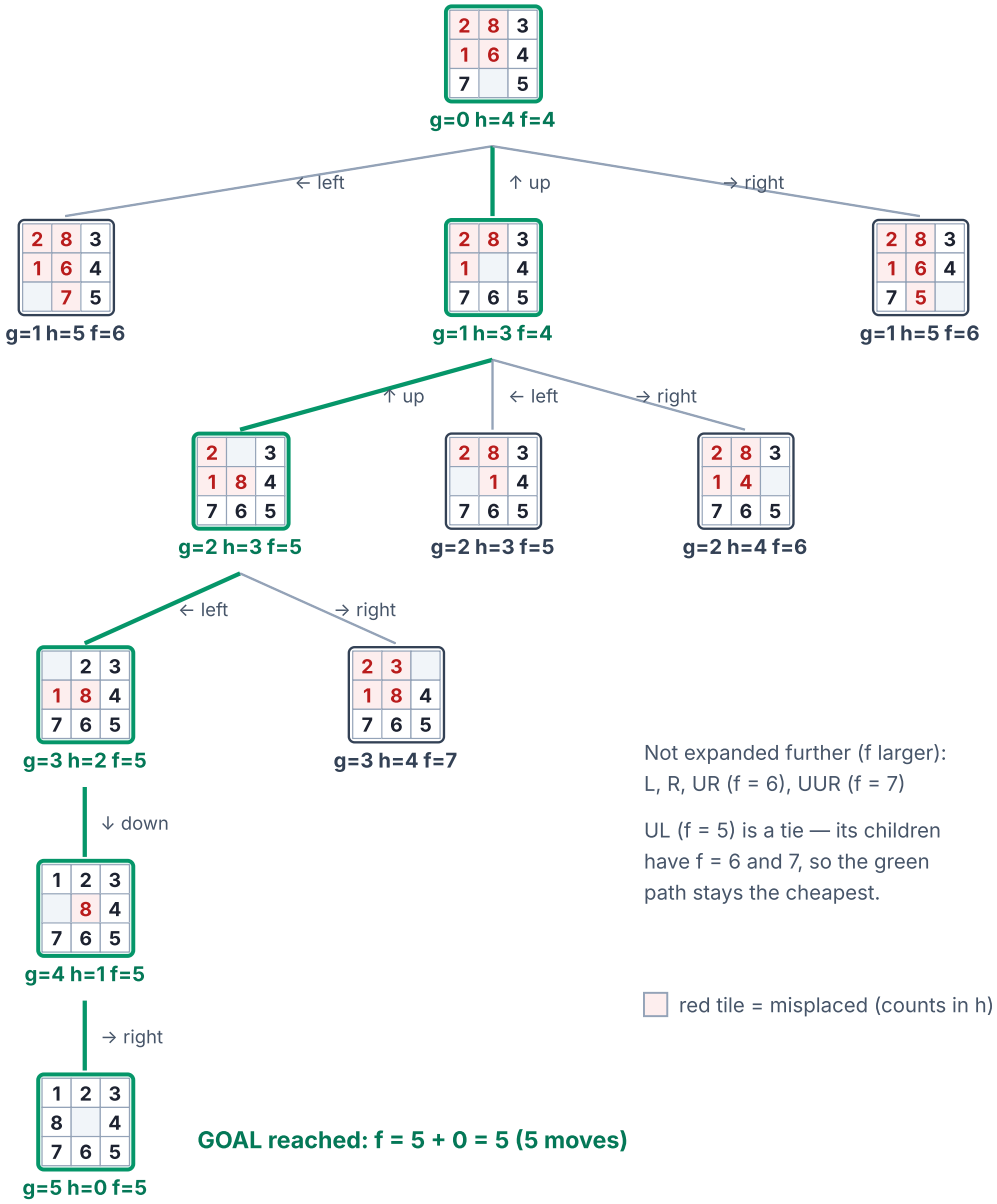
Goal

1	2	3
8		4
7	6	5

2 8 3 / 1 6 4 / 7 _ 5 → 1 2 3 / 8 _ 4 / 7 6 5

A* IN ONE LINE

Expand the node with the lowest f(n) = g(n) + h(n). g = moves made so far (depth); h = misplaced tiles (don't count the blank). Admissible h ⇒ A* finds the optimal (cheapest) path.



Search tree. Moves are named by where the blank goes. Green = chosen path; every board shows g, h, f.

Step	Expand (lowest f)	Children generated (f = g + h)	Open list after
0	Start: h = 4 (tiles 2, 8, 1, 6 misplaced), f = 0 + 4 = 4	L: 1+5 = 6, U: 1+3 = 4, R: 1+5 = 6	U(4), L(6), R(6)
1	U (f = 4)	UU: 2+3 = 5, UL: 2+3 = 5, UR: 2+4 = 6	UU(5), UL(5), L, R, UR(6)
2	UU (f = 5, tie → pick first)	UUL: 3+2 = 5, UUR: 3+4 = 7	UUL(5), UL(5), …
3	UUL (f = 5)	UULD: 4+1 = 5	UULD(5), UL(5), …
4	UULD (f = 5)	GOAL: 5+0 = 5	GOAL(5)
5	**Goal tested → stop.** Path: blank Up → Up → Left → Down → Right. **Cost = 5 moves** (optimal).		

EXAM TIPS FOR A*

- Write **f = g + h** and state h = misplaced tiles **excluding the blank** at the top.
- Under every board write g, h, f. Circle the node you expand next — that shows "lowest f first".
- Don't expand back to the parent state (repeated state).
- End with: path, number of moves, "optimal because h is admissible (never overestimates)".
- If they say h = Manhattan distance instead, only the h numbers change — method is identical.

REMEMBER IT

"Go = Gone + Guess." f (total) = g (steps **gone**) + h (**guess** to goal). Pick the smallest total every time — like choosing the auto with the lowest meter + remaining-distance estimate.

Q3. "If the team works overtime, they will finish on time. The team worked overtime. Therefore they finished on time." Check validity using propositional logic. Is overtime necessarily sufficient? SURE SHOT

Why: last year's Q1b + "propositional logic" on the insider list. Expect the same type with a new story (rain/match, study/pass, alarm/burglar).

Symbols: O = "the team works overtime", F = "the team finishes on time". **Premises:** P1: O ⇒ F, P2: O. **Conclusion:** F.

O	F	O ⇒ F (P1)	O (P2)	F (concl.)
T	T	T	T	**T ← only row with both premises true**
T	F	F	T	F
F	T	T	F	T
F	F	T	F	F

Valid — in every row where all premises are true, the conclusion is true. This is the inference rule **Modus Ponens**: $O \Rightarrow F,\ O \vdash F$. Equivalently $(O \Rightarrow F) \wedge O \Rightarrow F$ is a **tautology**.

Is overtime necessarily sufficient? No — validity is about form, not truth. The argument only shows: *if* premise 1 is true, the conclusion must follow. Whether overtime really guarantees completion is a real-world claim (scope creep, bugs, sick staff can still delay it). Valid + true premises = **sound**; we can't prove premise 1 by logic alone.

THE TRAP VARIANTS (LIKELY TWIST THIS YEAR)

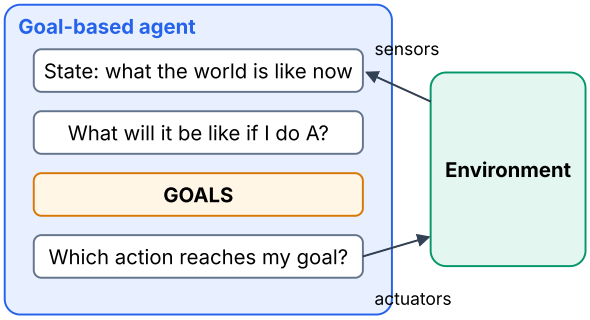
Form	Example	Valid?
Modus Ponens: P⇒Q, P ⊢ Q	Overtime ⇒ on time; overtime ⊢ on time	**Valid**
Modus Tollens: P⇒Q, ¬Q ⊢ ¬P	…; not on time ⊢ no overtime	**Valid**
Affirming the consequent: P⇒Q, Q ⊢ P	…; finished on time ⊢ they worked overtime	**Invalid (row O=F, F=T)**
Denying the antecedent: P⇒Q, ¬P ⊢ ¬Q	…; no overtime ⊢ not on time	**Invalid (row O=F, F=T)**

REMEMBER IT

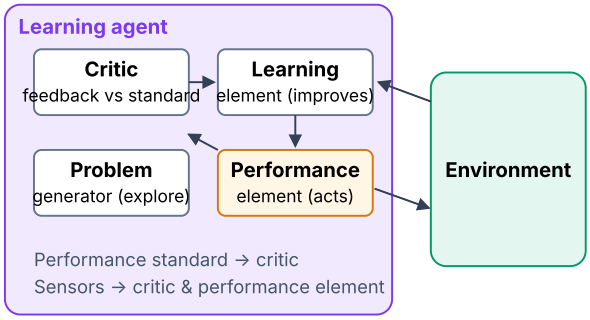
"P ⇒ Q is false only when Promise is broken": P true, Q false. For validity: **find a row where premises are all T but conclusion is F**. Found one → invalid. None → valid. Mnemonic for valid forms: "**Ponens Pushes forward, Tollens Turns back**".

Q4. Distinguish between a learning agent and a goal-based agent (with diagrams and an example). VERY LIKELY

Why: exact Q1a last year. This year they may swap the pair (simple reflex vs model-based, goal-based vs utility-based) — the table at the end covers every pair.



Goal-based: plans/searches for actions that reach a goal.



Learning: improves its own behaviour from feedback.

Basis	Goal-based agent	Learning agent
Idea	Chooses actions that achieve a desired **goal**, using search and planning	Improves its performance over time from **experience / feedback**
Key parts	State + model of the world + goal information	Performance element, critic, learning element, problem generator
Knowledge	Fixed — given by the designer	Grows — modifies itself; can start with little knowledge
Handles new situations	Only if the model covers them	Adapts to unknown environments
Question it asks	"Which action leads to my goal?"	"How can I do better next time?"
Example	Google Maps routing to a destination; taxi deciding a route to the airport	Spam filter improving from user "report spam"; taxi learning that braking hard on wet roads is bad

Note: any agent type (including goal-based) can be turned into a learning agent — the learning element improves its performance element.

Other pairs they may ask (one-line differences)	
Simple reflex vs model-based	Reflex acts only on the **current percept** with if-then rules (fails in partially observable worlds); model-based keeps an **internal state** using a model of how the world changes.
Goal-based vs utility-based	Goal: success is binary (reached / not); utility: a **utility function** scores how good each state is (fastest, safest, cheapest route) — chooses the best among many ways to reach the goal.

REMEMBER THE 5 AGENT TYPES IN ORDER (EACH ADDS ONE THING)

"Silly Monkeys Grab Useful Lemons" — Simple reflex (rules) → Model-based (+ memory/state) → Goal-based (+ goal) → Utility-based (+ happiness score) → Learning (+ improves itself). Learning parts: "**C-L-P-P**" = Critic, Learning element, Performance element, Problem generator.

Q5. Convert to CNF: (a) (A ∨ (B ∧ C)) ⇒ (¬D ∨ E) (b) (A ∨ (B ∧ (C ∨ ¬D))) ∧ (¬E ⇒ F)

VERY LIKELY

Why: last year's Q3a (7 marks) and part of every resolution question. Practise until it's mechanical.

4 STEPS (ALWAYS IN THIS ORDER)

① Remove ⇔ : P⇔Q → (P⇒Q) ∧ (Q⇒P) ② Remove ⇒ : P⇒Q → ¬P ∨ Q ③ Push ¬ inside (De Morgan, ¬¬P = P) ④ Distribute ∨ over ∧ : P ∨ (Q ∧ R) → (P ∨ Q) ∧ (P ∨ R)

(a)
(A ∨ (B ∧ C)) ⇒ (¬D ∨ E)
② ¬(A ∨ (B ∧ C)) ∨ (¬D ∨ E)
③ (¬A ∧ ¬(B ∧ C)) ∨ ¬D ∨ E
③ (¬A ∧ (¬B ∨ ¬C)) ∨ ¬D ∨ E
④ distribute (¬D ∨ E) over ∧:
(¬A ∨ ¬D ∨ E) ∧ (¬B ∨ ¬C ∨ ¬D ∨ E)

(b)
(A ∨ (B ∧ (C ∨ ¬D))) ∧ (¬E ⇒ F)
② ¬E ⇒ F → ¬¬E ∨ F → E ∨ F
④ A ∨ (B ∧ (C ∨ ¬D)) → (A ∨ B) ∧ (A ∨ C ∨ ¬D)
Combine:
(A ∨ B) ∧ (A ∨ C ∨ ¬D) ∧ (E ∨ F)
3 clauses: {A, B}, {A, C, ¬D}, {E, F}

PRACTICE VARIANT (BICONDITIONAL) — TRY IT, ANSWER BELOW
B ⇔ (P ∨ Q) → (¬B ∨ P ∨ Q) ∧ (¬P ∨ B) ∧ (¬Q ∨ B).

REMEMBER IT
"I-I-N-D" = "I Intend No Dumb mistakes": **I**ff out → **I**mplies out → **N**egation in → **D**istribute. De Morgan: "**break the line, change the sign**" (¬ over ∧ becomes ∨ and vice versa).

Q6. Classify the task environment of the given agents (fully/partially observable, deterministic/stochastic, etc.) IMPORTANT TOPIC

Why: usually attached to PEAS (sir likes scenarios). 5 marks = define dimensions + fill a table.

Dimension	Test question to ask
Fully vs partially observable	Can the sensors see the complete relevant state?
Single vs multi-agent	Are other agents affecting my performance? (competitive / cooperative)
Deterministic vs stochastic	Is the next state fully decided by the current state + my action?
Episodic vs sequential	Does the current decision affect future decisions?
Static vs dynamic	Can the world change while I'm thinking? (semi-dynamic: only the score/clock changes)
Discrete vs continuous	Finite distinct states/actions, or smooth values?
Known vs unknown	Does the agent know the rules/outcomes of its actions?

Task	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Taxi / self-driving car	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Wumpus world	Partially	Single	Deterministic	Sequential	Static	Discrete
Vacuum cleaner (2 rooms)	Partially (local dirt sensor)	Single	Deterministic	Episodic	Static	Discrete

REMEMBER IT
"**O-A-D-E-S-D-K**" → "**O**ld **A**gents **D**on't **E**at **S**picy **D**osa, **K**id" = Observable, Agents, Deterministic, Episodic, Static, Discrete, Known. Real world = the "hard" side of every pair.

Q7. Scenario: which search strategy would you use and why? (BFS / DFS / UCS / A*) IMPORTANT TOPIC

Why: sir said "problem solving, no pseudocode" — so search comes as "choose and justify", not "write the algorithm".

Scenario	Best choice	Justification
Find the route with the fewest metro stops	**BFS**	All steps cost the same (1 stop) → BFS gives the shallowest = optimal path; complete.
Delivery robot: roads have different lengths; need cheapest route; no distance estimate	**Uniform-cost search**	Expands lowest path cost g(n) first → optimal for unequal costs.
Google-Maps-like routing where straight-line distance to destination is known	**A***	f = g + h uses the admissible straight-line heuristic → optimal and far fewer nodes than UCS.
Solving a maze on a phone with very little memory; depth limited	**DFS / Iterative deepening**	DFS memory O(bm); IDS adds completeness and optimality (equal costs) with O(bd) memory.
Need a quick, "good enough" answer, optimality not required	**Greedy best-first**	f = h only → fast, but not optimal (Arad→Bucharest gives 450 km instead of 418).
Start and goal both known, uniform costs (e.g., friend-of-friend link)	**Bidirectional**	Searches from both ends: $O(b^{d/2})$ instead of $O(b^d)$.

Property	BFS	UCS	DFS	IDS	Greedy	A*
Complete?	Yes	Yes	No	Yes	No	Yes
Optimal?	Equal costs	Yes	No	Equal costs	No	Yes (admissible h)
Uses	queue	g(n)	stack	depth limits	h(n)	g(n)+h(n)

REMEMBER IT

BFS = **B**readth = layer by layer ("ripples in a pond"). **D**FS = **D**ive deep ("go down one corridor first"). UCS = "**U**ber **C**heapest **S**o far". Greedy = "looks only ahead". A* = "looks back (g) **and** ahead (h)".

UNIT I — ALSO GLANCE AT (1-LINE ANSWERS)

Rational agent = does the action that maximises expected performance given its percept sequence and knowledge (rational ≠ omniscient). Turing test = interrogator can't tell machine from human (needs NLP, knowledge representation, reasoning, learning). Admissible h = never overestimates; consistent $h(n) \le c(n, n') + h(n')$.